

Deadlock-02

Banker's Algorithm

- Multiple instances
- Each process must a priori claim maximum use
- When a process requests a resource it may have to wait
- When a process gets all its resources it must return them in a finite amount of time

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If available $[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i,j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i,j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Safety Algorithm

1. Let *Work* and *Finish* be vectors of length m and n , respectively. Initialize:

Work = *Available*

Finish [i] = *false* for $i = 0, 1, \dots, n-1$

2. Find an i such that both:

(a) *Finish* [i] = *false*

(b) $Need_i \leq Work$

If no such i exists, go to step 4

3. $Work = Work + Allocation_i$
Finish [i] = *true*
go to step 2

4. If *Finish* [i] == *true* for all i , then the system is in a safe state

Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;

3 resource types:

A (10 instances), B (5 instances), and C (7 instances)

Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example (Cont.)

- The content of the matrix *Need* is defined to be *Max – Allocation*

	<u><i>Need</i></u>		
	<i>A</i>	<i>B</i>	<i>C</i>
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Safety Algorithm Example

Step 1 of Safety Algo

$m=3, n=5$
 Work = Available

Work =	3	3	2		
	0	1	2	3	4

Finish =	false	false	false	false	false
----------	-------	-------	-------	-------	-------

Step 2

For $i=0$
 Need₀ = 7, 4, 3
 Finish [0] is false and Need₀ > Work
 So P₀ must wait

Step 2

For $i=1$
 Need₁ = 1, 2, 2
 Finish [1] is false and Need₁ < Work
 So P₁ must be kept in safe sequence

Step 3

Work = Work + Allocation₁

Work =	5	3	2		
	0	1	2	3	4

Finish =	false	true	false	false	false
----------	-------	------	-------	-------	-------

Step 2

For $i=2$
 Need₂ = 6, 0, 0
 Finish [2] is false and Need₂ > Work
 So P₂ must wait

Step 2

For $i=3$
 Need₃ = 0, 1, 1
 Finish [3] = false and Need₃ < Work
 So P₃ must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =	7	4	3		
	0	1	2	3	4

Finish =	false	true	false	true	false
----------	-------	------	-------	------	-------

Step 2

For $i=4$
 Need₄ = 4, 3, 1
 Finish [4] = false and Need₄ < Work
 So P₄ must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =	7	4	5		
	0	1	2	3	4

Finish =	false	true	false	true	true
----------	-------	------	-------	------	------

Step 2

For $i=0$
 Need₀ = 7, 4, 3
 Finish [0] is false and Need₀ < Work
 So P₀ must be kept in safe sequence

Step 3

Work = Work + Allocation₀

Work =	7	5	5		
	0	1	2	3	4

Finish =	true	true	false	true	true
----------	------	------	-------	------	------

Step 2

For $i=2$
 Need₂ = 6, 0, 0
 Finish [2] is false and Need₂ < Work
 So P₂ must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =	10	5	7		
	0	1	2	3	4

Finish =	true	true	true	true	true
----------	------	------	------	------	------

Finish [i] = true for $0 \leq i \leq n$
 Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

Resource-Request Algorithm for Process P_i

Request = request vector for process P_i . If $Request_i[j] = k$ then process P_i wants k instances of resource type R_j

1. If $Request_i \leq Need_i$, go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $Request_i \leq Available$, go to step 3. Otherwise P_i must wait, since resources are not available
3. Pretend to allocate requested resources to P_i by modifying the state as follows:

Available = *Available* - *Request*;

*Allocation*_{*i*} = *Allocation*_{*i*} + *Request*_{*i*};

*Need*_{*i*} = *Need*_{*i*} - *Request*_{*i*};

- If safe $\Rightarrow$ the resources are allocated to P_i
- If unsafe $\Rightarrow P_i$ must wait, and the old resource-allocation state is restored

Example: P_1 Request (1,0,2)

$\begin{matrix} & A & B & C \\ \text{Request}_1 = & 1, & 0, & 2 \end{matrix}$

To decide whether the request is granted we use Resource Request algorithm

$\begin{matrix} 1, 0, 2 & 1, 2, 2 \\ \text{Request}_1 < \text{Need}_1 \end{matrix}$

Step 1

$\begin{matrix} 1, 0, 2 & 3, 3, 2 \\ \text{Request}_1 < \text{Available} \end{matrix}$

Step 2

Step 3

$\text{Available} = \text{Available} - \text{Request}_1$
 $\text{Allocation}_1 = \text{Allocation}_1 + \text{Request}_1$
 $\text{Need}_1 = \text{Need}_1 - \text{Request}_1$

Process	Allocation	Need	Available
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

Example: P_1 Request (1,0,2)(cont.)

- We must determine whether this new system state is safe. To do so, we again execute Safety algorithm on the above data structures.(Showed in next slide)

Step 1 of Safety Algo

m=3, n=5
 Work = Available

Work =

2	3	0
---	---	---

0 1 2 3 4

Finish =

false	false	false	false	false
-------	-------	-------	-------	-------

Step 2:

For i = 0
 Need₀ = 7, 4, 3
 Finish [0] is false and Need₀ > Work
 So P₀ must wait

Step 2:

For i = 1
 Need₁ = 0, 2, 0
 Finish [1] is false and Need₁ < Work
 So P₁ must be kept in safe sequence

Step 3

Work = Work + Allocation₁

Work =

5	3	2
---	---	---

0 1 2 3 4

Finish =

false	true	false	false	false
-------	------	-------	-------	-------

Step 2:

For i = 2
 Need₂ = 6, 0, 0
 Finish [2] is false and Need₂ > Work
 So P₂ must wait

Step 2:

For i = 3
 Need₃ = 0, 1, 1
 Finish [3] = false and Need₃ < Work
 So P₃ must be kept in safe sequence

Step 3

Work = Work + Allocation₃

Work =

7	4	3
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	false
-------	------	-------	------	-------

Step 2:

For i = 4
 Need₄ = 4, 3, 1
 Finish [4] = false and Need₄ < Work
 So P₄ must be kept in safe sequence

Step 3

Work = Work + Allocation₄

Work =

7	4	5
---	---	---

0 1 2 3 4

Finish =

false	true	false	true	true
-------	------	-------	------	------

Step 2:

For i = 0
 Need₀ = 7, 4, 3
 Finish [0] is false and Need < Work
 So P₀ must be kept in safe sequence

Step 3

Work = Work + Allocation₀

Work =

7	5	5
---	---	---

0 1 2 3 4

Finish =

true	true	false	true	true
------	------	-------	------	------

Step 2:

For i = 2
 Need₂ = 6, 0, 0
 Finish [2] is false and Need₂ < Work
 So P₂ must be kept in safe sequence

Step 3

Work = Work + Allocation₂

Work =

10	5	7
----	---	---

0 1 2 3 4

Finish =

true	true	true	true	true
------	------	------	------	------

Step 4

Finish [i] = true for 0 ≤ i ≤ n
 Hence the system is in Safe state

The safe sequence is P₁, P₃, P₄, P₀, P₂

Hence the new system state is safe, so we can immediately grant the request for process P₁.

Exercise

- Can request for $(3,3,0)$ by P_4 be granted?
- Can request for $(0,2,0)$ by P_0 be granted?

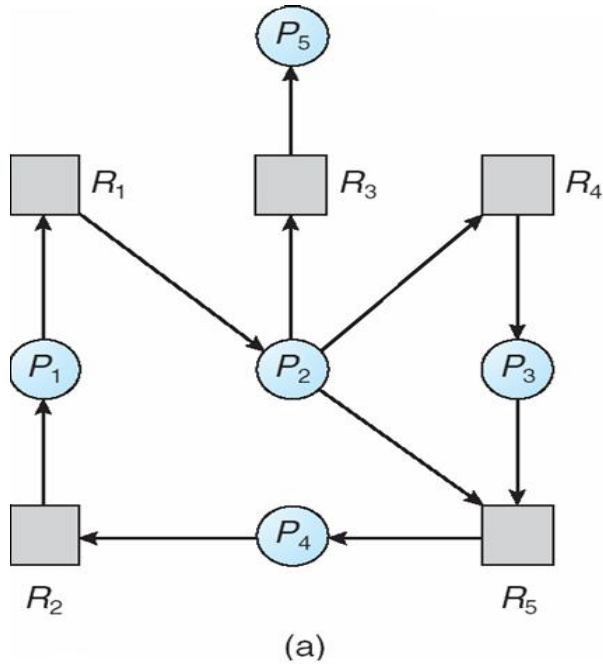
Deadlock Detection

- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

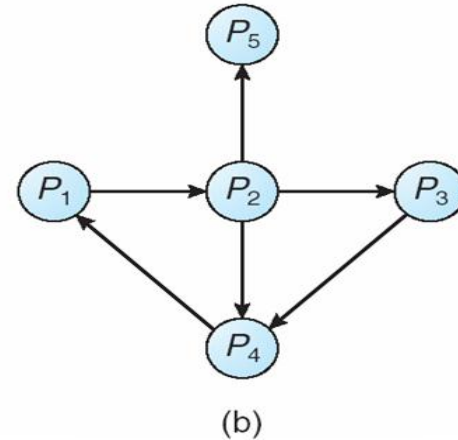
Single Instance of Each Resource Type

- Maintain *wait-for* graph
 - Nodes are processes
 - $P_i \rightarrow P_j$ if P_i is waiting for P_j
- Periodically invoke an algorithm that searches for a cycle in the graph. If there is a cycle, there exists a deadlock
- An algorithm to detect a cycle in a graph requires an order of n^2 operations, where n is the number of vertices in the graph

Resource-Allocation Graph and Wait-for Graph



Resource-Allocation Graph



Corresponding wait-for graph

Several Instances of a Resource Type

- **Available:** A vector of length m indicates the number of available resources of each type.
- **Allocation:** An $n \times m$ matrix defines the number of resources of each type currently allocated to each process.
- **Request:** An $n \times m$ matrix indicates the current request of each process. If $Request[i_j] = k$, then process P_i is requesting k more instances of resource type R_j .

Detection Algorithm

1. Let *Work* and *Finish* be vectors of length m and n , respectively Initialize:

(a) *Work* = *Available*

(b) For $i = 1, 2, \dots, n$, if $Allocation_i \neq 0$, then
 $Finish[i] = false$; otherwise, $Finish[i] = true$

2. Find an index i such that both:

(a) $Finish[i] == false$

(b) $Request_i \leq Work$

If no such i exists, go to step 4

Detection Algorithm (Cont.)

3. $Work = Work + Allocation_i$
 $Finish[i] = true$
go to step 2
4. If $Finish[i] == false$, for some $i, 1 \leq i \leq n$, then the system is in deadlock state. Moreover, if $Finish[i] == false$, then P_i is deadlocked

Algorithm requires an order of $O(m \times n^2)$ operations to detect whether the system is in deadlocked state

Example of Detection Algorithm

- Five processes P_0 through P_4 ; three resource types A (7 instances), B (2 instances), and C (6 instances)
- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

Example of Detection Algorithm(Cont.)

1. In this, $Work = [0, 0, 0]$ & $Finish = [false, false, false, false, false]$
 2. $i=0$ is selected as both $Finish[0] = false$ and $[0, 0, 0] \leq [0, 0, 0]$.
 $Work = [0, 0, 0] + [0, 1, 0] \Rightarrow [0, 1, 0]$ & $Finish = [true, false, false, false, false]$.
 3. $i=2$ is selected as both $Finish[2] = false$ and $[0, 0, 0] \leq [0, 1, 0]$.
 $Work = [0, 1, 0] + [3, 0, 3] \Rightarrow [3, 1, 3]$ & $Finish = [true, false, true, false, false]$.
 4. $i=1$ is selected as both $Finish[1] = false$ and $[2, 0, 2] \leq [3, 1, 3]$.
 $Work = [3, 1, 3] + [2, 0, 0] \Rightarrow [5, 1, 3]$ & $Finish = [true, true, true, false, false]$.
 5. $i=3$ is selected as both $Finish[3] = false$ and $[1, 0, 0] \leq [5, 1, 3]$.
 $Work = [5, 1, 3] + [2, 1, 1] \Rightarrow [7, 2, 4]$ & $Finish = [true, true, true, true, false]$.
 6. $i=4$ is selected as both $Finish[4] = false$ and $[0, 0, 2] \leq [7, 2, 4]$.
 $Work = [7, 2, 4] + [0, 0, 2] \Rightarrow [7, 2, 6]$ & $Finish = [true, true, true, true, true]$.
- Sequence $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ will result in $Finish[i] = true$ for all i
- Since $Finish$ is a vector of all true it means **there is no deadlock** in this example.

Example of Detection Algorithm(Cont.)

- P_2 requests an additional instance of type C

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	1
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- State of system?
 - Can reclaim resources held by process P_0 , but insufficient resources to fulfill other processes; requests
 - Deadlock exists, consisting of processes P_1 , P_2 , P_3 , and P_4

Detection-Algorithm Usage

- When, and how often, to invoke depends on:
 - How often a deadlock is likely to occur?
 - How many processes will need to be rolled back?
 - one for each disjoint cycle
- If detection algorithm is invoked arbitrarily, there may be many cycles in the resource graph and so we would not be able to tell which of the many deadlocked processes “caused” the deadlock.

Recovery from Deadlock: Process Termination

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
 - Priority of the process
 - How long process has computed, and how much longer to completion
 - Resources the process has used
 - Resources process needs to complete
 - How many processes will need to be terminated
 - Is process interactive or batch?

Recovery from Deadlock: Resource Preemption

- Selecting a victim – minimize cost
- Rollback – return to some safe state, restart process for that state
- Starvation – same process may always be picked as victim, include number of rollback in cost factor

THANK YOU!